

Lab 11: Automation DevOps

Lab 11: Automation, DevOps, and Lifecycle Management

Lab summary

These labs teach PBIP source control and introduce optional automation patterns for validated environments.

Concept note: Lifecycle management goals

A production Power BI lifecycle should be repeatable, reviewable, governable, and recoverable. Use PBIP and git to make changes visible, support promotion between environments, and preserve rollback evidence when a deployment needs to be reversed.

Novice-friendly how-to guide

Review PBIP source structure

1. Open the PBIP project folder in File Explorer or VS Code.
2. Locate the `.pbip` file.
3. Locate the `.Report` folder and `.SemanticModel` folder.
4. Open a few JSON or TMDL files as text to understand what is source-controlled.
5. Do not edit generated files unless the lab explicitly tells you to.

Practice a basic git workflow

1. Open a terminal at the repo root.
2. Run `git status`.
3. Create or switch to the working branch chosen for the lab.
4. Make a small documented change.
5. Run `git diff` to review it.
6. Commit only the intended files.

Azure Government readiness

PBIP and local git workflows are **Gov-ready**. Tabular Editor, ALM Toolkit, REST APIs, PowerShell, service principals, XMLA, Fabric Git integration, Azure DevOps, and GitHub Actions are **Verify for Gov** or **Commercial-focused / Verify for Gov**.

Prerequisites

- Git installed.
- PBIP project from earlier modules.
- Optional approved external tools.
- Optional API/PowerShell permissions.
- Optional service principal.
- Optional Azure DevOps or GitHub repository.

Exercise 1: PBIP file structure

Objective: Understand what gets checked into source control.

Tasks

1. Open a PBIP project folder.
2. Identify report and semantic model folders.
3. Review definition files.
4. Identify generated or binary files that should not be the source of record.
5. Document the PBIP structure.

Expected result

Learners understand why PBIP supports code review better than PBIX.

Exercise 2: Source control workflow

Objective: Track and review Power BI changes with git.

Tasks

1. Create a feature branch.
2. Make a small report or model change.
3. Review `git status`.
4. Review changed files.
5. Commit the change.
6. Discuss pull request review expectations.

Expected result

Learners can use a basic source-control workflow for PBIP content.

Concept note: Branches, pull requests, and releases

Branches isolate work, commits create reviewable checkpoints, pull requests focus peer review on intentional model/report changes, and tags or release notes help operations teams identify what was promoted and how to roll back.

Exercise 3: Tabular Editor workflow where available

Azure Government note: Tabular Editor is **Verify for Gov**. Validate workstation policy, XMLA, tenant settings, and customer approval.

Tasks when available

1. Open the model with Tabular Editor.
2. Review measures and tables.
3. Make a small metadata change.
4. Save and review git diff.

Exercise 4: ALM Toolkit model comparison where available

Azure Government note: ALM Toolkit is **Verify for Gov**. Validate workstation policy, XMLA, tenant settings, and customer approval.

Tasks when available

1. Compare two model versions.
2. Review table, relationship, and measure differences.
3. Discuss deployment impact.

Optional lab: Power BI REST API deployment

Azure Government note: REST APIs are **Verify for Gov**. Validate endpoint, permission, tenant settings, and authentication.

Conceptual tasks

1. Identify target workspace.
2. Identify authentication method.
3. Identify deployment API operations.
4. Document validation requirements.

Optional lab: PowerShell administration

Azure Government note: PowerShell administration is **Verify for Gov**. Validate module support, endpoint, permissions, and customer policy.

Conceptual tasks

1. Review common admin commands.
2. Identify required permissions.
3. Document endpoint and cloud validation.

Optional lab: Service principal authentication

Azure Government note: Service principals are **Verify for Gov**. Requires Entra ID app registration, tenant setting approval, security group scoping, and workspace access.

Conceptual tasks

1. Review app registration requirements.
2. Review tenant setting requirements.
3. Review workspace access.
4. Review secret/certificate handling.

Optional commercial lab: Fabric workspace Git integration

Azure Government note: Fabric workspace Git integration is **Commercial-focused / Verify for Gov**. Do not require in Gov labs unless confirmed.

Tasks when available

1. Connect workspace to git.
2. Select branch and folder.
3. Review sync status.
4. Review branching and workspace sync behavior.
5. Commit workspace changes.

Concept note: Workspace-connected Git caveats

Fabric workspace Git integration changes where synchronization happens: workspace items can connect directly to a branch and folder. Validate branching behavior, item support, sync direction, conflict handling, and Commercial/Gov availability before relying on it for release management.

Exercise 5: Conceptual CI/CD pipeline for Azure DevOps

Objective: Design a deployment pipeline pattern.

Tasks

1. Review the conceptual Azure DevOps pipeline.
2. Identify validation stage.
3. Identify authentication approach.
4. Identify deployment stage.
5. Add Gov validation notes.

Exercise 6: Conceptual CI/CD pipeline for GitHub Actions

Objective: Design a GitHub Actions deployment pattern.

Tasks

1. Review the conceptual GitHub Actions pipeline.
2. Identify trigger.
3. Identify validation stage.
4. Identify authentication approach.
5. Add Gov validation notes.

Concept note: Azure Government automation boundaries

For Azure Government delivery, validate cloud endpoints, identity authority, network path, tenant settings, approved tooling, and feature availability before using APIs, service principals, XMLA, Fabric Git integration, or CI/CD runners.

Exercise 7: Deployment checklist

Objective: Validate readiness before deployment.

Tasks

1. Complete the source control section.
2. Complete model/report validation.
3. Complete environment configuration.
4. Complete automation validation.
5. Complete Azure Government validation.
6. Record release evidence.

Validation checklist

- ☐ Lifecycle goals and rollback evidence documented.
- ☐ PBIP structure reviewed.
- ☐ Git workflow, branch, pull request, and release expectations documented.
- ☐ External tools marked **Verify for Gov.**
- ☐ REST APIs marked **Verify for Gov.**
- ☐ PowerShell marked **Verify for Gov.**
- ☐ Service principals marked **Verify for Gov.**
- ☐ Fabric Git integration branching and sync caveats marked **Commercial-focused / Verify for Gov.**
- ☐ Azure DevOps and GitHub Actions marked **Verify for Gov.**
- ☐ Azure Government endpoints, identity, network, and policy validation documented.
- ☐ Deployment checklist completed.